# STAT 639 - Homework 2

ISLR Chapter 4: Exercises 1, 4, 8, 14; Chapter 9: Exercise 3(a)-(f)

Bao Dang

October 04, 2026

## Contents

**1 Chapter 4 - Exercise 1** **2**

**2 Chapter 4 - Exercise 4** **2**
2.1 (a) One feature . . . 2
2.2 (b) Two features . . . 3
2.3 (c) One hundred features . . . 3
2.4 (d) Interpretation . . . 3
2.5 (e) Hypercube containing 10% of observations . . . 3

**3 Chapter 4 - Exercise 8** **4**

**4 Chapter 4 - Exercise 14** **4**
4.1 (a) Create `mpg01` . . . 4
4.2 (b) Explore associations . . . 5
4.3 (c) Split into training and test sets . . . 7
4.4 (d) LDA . . . 8
4.5 (e) QDA . . . 8
4.6 (f) Logistic regression . . . 8
4.7 (g) Naive Bayes . . . 9
4.8 (h) KNN for several values of K . . . 9
4.8.1 Model comparison . . . 10

**5 Chapter 9 - Exercise 3** **11**
5.1 (a) Sketch the observations . . . 11
5.2 (b) Optimal separating hyperplane . . . 12
5.3 (c) Classification rule . . . 14
5.4 (d) Margin . . . 14
5.5 (e) Support vectors . . . 15
5.6 (f) Moving observation 7 slightly . . . 16

# 1 Chapter 4 - Exercise 1

The logistic regression model can be written as

$$p(X) = \frac{e^{\beta_0+\beta_1 X}}{1+e^{\beta_0+\beta_1 X}}.$$

Let $\eta = \beta_0 + \beta_1 X$. Then

$$p(X) = \frac{e^{\eta}}{1+e^{\eta}}.$$

Multiplying both sides by $1 + e^{\eta}$,

$$p(X)(1+e^{\eta}) = e^{\eta}.$$

Therefore,

$$p(X) = e^{\eta}(1-p(X)),$$

so

$$\frac{p(X)}{1-p(X)} = e^{\eta} = e^{\beta_0+\beta_1 X}.$$

Taking the natural logarithm gives

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X.$$

Thus the logistic-function representation and the logit representation are equivalent.

# 2 Chapter 4 - Exercise 4

This exercise illustrates the **curse of dimensionality** for local methods such as KNN.

## 2.1 (a) One feature

For a test observation $x$ away from the edges ($0.05 \le x \le 0.95$), the neighborhood is $[x - 0.05,\, x + 0.05]$. Because $X$ is uniform on $[0, 1]$, this interval contains 10% of the observations.

Near the edges the interval is truncated. For $x < 0.05$ it is $[0,\, x + 0.05]$, with length $x + 0.05$; the case $x > 0.95$ is symmetric. Averaging over all test points,

$$\int_{0.05}^{0.95} 0.1\,dx \;+\; 2\int_{0}^{0.05} (x+0.05)\,dx = 0.09 + 0.0075 = 0.0975.$$

So on average about **9.75%** of the observations are used. Ignoring the edge effect gives the approximation of 10%.

```
f <- function(x) pmin(x + 0.05, 1) - pmax(x - 0.05, 0)
integrate(f, 0, 1)$value
```

```
## [1] 0.09749969
```

## 2.2 (b) Two features

An observation must fall within 10% of the range for both $X_1$ and $X_2$. Because $X_1$ and $X_2$ are independent and uniform, the fractions multiply:

$$0.0975^2 \approx 0.0095 \approx 0.95\%,$$

or $0.10^2 = 1\%$ if edge effects are ignored.

## 2.3 (c) One hundred features

For $p = 100$, the expected fraction is

$$0.0975^{100} \approx 7.9 \times 10^{-102},$$

or $0.10^{100} = 10^{-100}$ if edge effects are ignored. Either way, this is essentially zero.

```
c(with_edges = 0.0975^100, ignoring_edges = 0.1^100)
```

```
##     with_edges ignoring_edges
##  7.951729e-102  1.000000e-100
```

## 2.4 (d) Interpretation

As the number of predictors increases, the fraction of observations that are simultaneously close to a test observation in every dimension decreases extremely quickly. Therefore, in high dimensions, KNN has very few genuinely nearby training observations available for prediction. To obtain enough neighbors, the neighborhood must become much larger, so the observations are no longer truly local.

## 2.5 (e) Hypercube containing 10% of observations

If each side of a $p$-dimensional hypercube has length $s$, its volume is $s^p$. We want

$$s^p = 0.10,$$

so

$$s = 0.10^{1/p}.$$

```
p <- c(1, 2, 100)
side_length <- 0.1^(1 / p)
data.frame(p = p, side_length = side_length)
```

```
##     p side_length
## 1   1   0.1000000
## 2   2   0.3162278
## 3 100   0.9772372
```

Hence:

- $p = 1$: $s = 0.10$
- $p = 2$: $s = \sqrt{0.10} \approx 0.316$
- $p = 100$: $s = 0.10^{1/100} \approx 0.977$

For 100 predictors, the neighborhood must span about 97.7% of the range of **each** predictor just to contain 10% of the observations on average. This shows why the idea of a "local" neighborhood breaks down in high dimensions.

# 3 Chapter 4 - Exercise 8

The logistic regression test error is given directly as **30%**.

For 1-nearest neighbors, the training error is 0% because each training observation is its own nearest neighbor. Let $E_{test}$ be the KNN test error. Because the training and test sets have equal size and the average error across both sets is 18%,

$$\frac{0 + E_{test}}{2} = 0.18.$$

Therefore,

$$E_{test} = 0.36.$$

So the 1-NN test error is **36%**, compared with **30%** for logistic regression. We should prefer **logistic regression** for classifying new observations because test-set performance is the relevant measure of generalization.

# 4 Chapter 4 - Exercise 14

This exercise uses the `Auto` data from the `ISLR2` package.

```
if (!requireNamespace("ISLR2", quietly = TRUE)) {
  install.packages("ISLR2")
}
if (!requireNamespace("MASS", quietly = TRUE)) {
  install.packages("MASS")
}
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071")
}
if (!requireNamespace("class", quietly = TRUE)) {
  install.packages("class")
}

library(ISLR2)
library(MASS)
library(e1071)
library(class)

data("Auto")
```

## 4.1 (a) Create `mpg01`

```
mpg_median <- median(Auto$mpg)
mpg01 <- ifelse(Auto$mpg > mpg_median, 1, 0)

auto <- data.frame(mpg01 = factor(mpg01), Auto)
```

```r
table(auto$mpg01)
```

```
##
##   0   1
## 196 196
```

```r
mpg_median
```

```
## [1] 22.75
```

`mpg01 = 1` represents cars with mileage above the sample median, while `mpg01 = 0` represents cars at or below the median.

## 4.2 (b) Explore associations

The candidate features in the `Auto` data are:

- `cylinders`: number of engine cylinders (between 3 and 8);
- `displacement`: engine displacement, in cubic inches;
- `horsepower`: engine horsepower;
- `weight`: vehicle weight, in pounds;
- `acceleration`: time to accelerate from 0 to 60 mph, in seconds (larger values mean slower acceleration);
- `year`: model year (1970 to 1982);
- `origin`: where the car was made (1 = American, 2 = European, 3 = Japanese).

The `name` variable (car name) is an identifier and is not used.

```r
par(mfrow = c(2, 3))

plot(auto$mpg01, auto$cylinders,
     xlab = "mpg01", ylab = "Cylinders",
     main = "Cylinders vs mpg01")

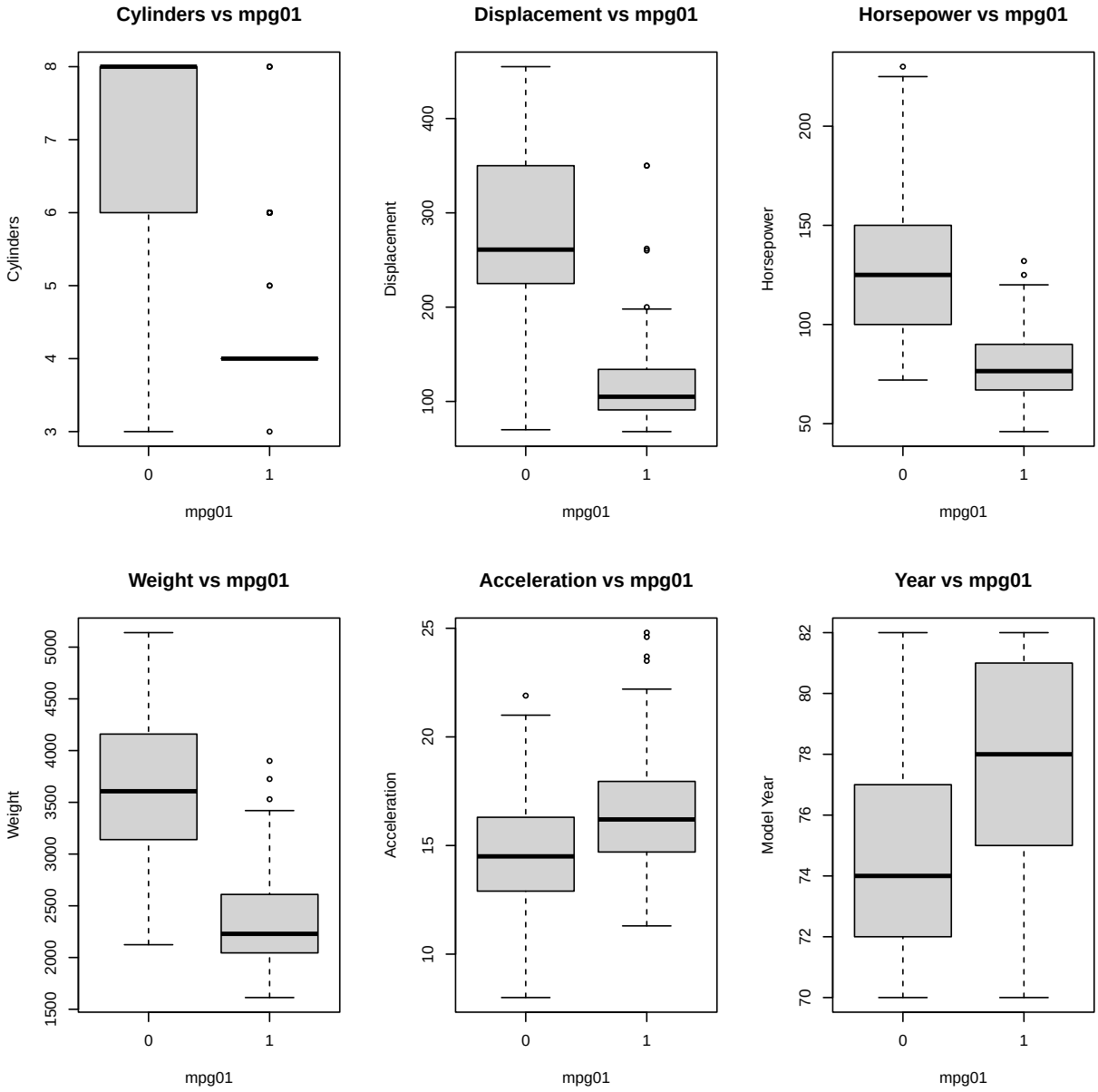
plot(auto$mpg01, auto$displacement,
     xlab = "mpg01", ylab = "Displacement",
     main = "Displacement vs mpg01")

plot(auto$mpg01, auto$horsepower,
     xlab = "mpg01", ylab = "Horsepower",
     main = "Horsepower vs mpg01")

plot(auto$mpg01, auto$weight,
     xlab = "mpg01", ylab = "Weight",
     main = "Weight vs mpg01")

plot(auto$mpg01, auto$acceleration,
     xlab = "mpg01", ylab = "Acceleration",
     main = "Acceleration vs mpg01")

plot(auto$mpg01, auto$year,
     xlab = "mpg01", ylab = "Model Year",
     main = "Year vs mpg01")
```




```
par(mfrow = c(1, 1))
```

Because `origin` is categorical, it is summarized with a table of proportions (each column sums to 1):

```
origin_label <- factor(auto$origin, levels = 1:3,
                       labels = c("American", "European", "Japanese"))
round(prop.table(table(mpg01 = auto$mpg01, Origin = origin_label), margin = 2), 2)
```

```
##      Origin
## mpg01 American European Japanese
##     0     0.71     0.21     0.11
##     1     0.29     0.79     0.89
```

Group medians for the numeric features:

```r
num_vars <- c("cylinders", "displacement", "horsepower",
              "weight", "acceleration", "year")
med <- sapply(num_vars, function(v) tapply(auto[[v]], auto$mpg01, median))
rownames(med) <- c("mpg01 = 0 (low mileage)", "mpg01 = 1 (high mileage)")
med
```

```
##                          cylinders displacement horsepower weight acceleration
## mpg01 = 0 (low mileage)          8          261      125.0   3607         14.5
## mpg01 = 1 (high mileage)         4          105       76.5   2229         16.2
##                          year
## mpg01 = 0 (low mileage)    74
## mpg01 = 1 (high mileage)   78
```

The boxplots and medians show clear differences between the two groups:

- **Engine and vehicle size.** Low-mileage cars have a median of 8 cylinders, compared with 4 for high-mileage cars. Their median engine displacement is 261 cubic inches, compared with 105. Their median horsepower is 125, compared with 76.5. Their median vehicle weight is 3607 lbs, compared with 2229 lbs. For displacement, horsepower, and weight, the boxes for the two groups barely overlap, so each of these features separates the classes well.
- **Model year.** Newer cars are more likely to have high mileage: the median model year is 1974 for low-mileage cars and 1978 for high-mileage cars. However, both groups span the full 1970–1982 range, so the boxes overlap considerably.
- **Acceleration.** High-mileage cars take slightly longer to reach 60 mph, with medians of 16.2 vs. 14.5 seconds. The overlap is large, so this feature is a weak predictor on its own.
- **Origin.** Most American cars are in the low-mileage group, while most European and Japanese cars are in the high-mileage group. Much of this, however, reflects the fact that American cars of this period were larger and heavier.

Cylinders, displacement, horsepower, and weight show the strongest separation, so these four are used as predictors below. They all measure engine and vehicle size, and they are highly correlated with one another. Year and origin are also associated with mileage, but their class distributions overlap more. They could be added in a further analysis; here the predictor set is kept to the four features with the clearest separation.

## 4.3 (c) Split into training and test sets

A reproducible 70/30 random split is used.

```r
set.seed(639)

train_id <- sample(
  seq_len(nrow(auto)),
  size = floor(0.70 * nrow(auto))
)

test_id <- setdiff(seq_len(nrow(auto)), train_id)

train <- auto[train_id, ]
test <- auto[test_id, ]

c(training = nrow(train), test = nrow(test))
```

```
## training     test
##      274      118
```

## 4.4 (d) LDA

```
lda_fit <- lda(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train
)

lda_pred <- predict(lda_fit, newdata = test)$class
lda_cm <- table(Predicted = lda_pred, Actual = test$mpg01)
lda_error <- mean(lda_pred != test$mpg01)

lda_cm
```

```
##          Actual
## Predicted  0  1
##         0 53  1
##         1 11 53
```

```
lda_error
```

```
## [1] 0.1016949
```

The LDA test error is 0.1017 (about 10.2%), i.e. 12 of 118 test cars are misclassified. Most of these errors (11 of 12) are low-mileage cars predicted to be high-mileage.

## 4.5 (e) QDA

```
qda_fit <- qda(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train
)

qda_pred <- predict(qda_fit, newdata = test)$class
qda_cm <- table(Predicted = qda_pred, Actual = test$mpg01)
qda_error <- mean(qda_pred != test$mpg01)

qda_cm
```

```
##          Actual
## Predicted  0  1
##         0 55  2
##         1  9 52
```

```
qda_error
```

```
## [1] 0.09322034
```

The QDA test error is 0.0932 (about 9.3%), i.e. 11 of 118 test cars are misclassified.

## 4.6 (f) Logistic regression

```
logit_fit <- glm(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train,
  family = binomial
)
```

```r
logit_prob <- predict(logit_fit, newdata = test, type = "response")
logit_pred <- factor(ifelse(logit_prob > 0.5, 1, 0), levels = levels(test$mpg01))

logit_cm <- table(Predicted = logit_pred, Actual = test$mpg01)
logit_error <- mean(logit_pred != test$mpg01)

logit_cm
```

```
##          Actual
## Predicted  0  1
##         0 53  1
##         1 11 53
```

```r
logit_error
```

```
## [1] 0.1016949
```

The logistic-regression test error is 0.1017 (about 10.2%), i.e. 12 of 118 test cars are misclassified.

## 4.7 (g) Naive Bayes

```r
nb_fit <- naiveBayes(
  mpg01 ~ cylinders + displacement + horsepower + weight,
  data = train
)

nb_pred <- predict(nb_fit, newdata = test)
nb_cm <- table(Predicted = nb_pred, Actual = test$mpg01)
nb_error <- mean(nb_pred != test$mpg01)

nb_cm
```

```
##          Actual
## Predicted  0  1
##         0 53  1
##         1 11 53
```

```r
nb_error
```

```
## [1] 0.1016949
```

The naive-Bayes test error is 0.1017 (about 10.2%), i.e. 12 of 118 test cars are misclassified.

## 4.8 (h) KNN for several values of K

KNN is distance-based, so the predictors are standardized using the **training-set** means and standard deviations. The same transformation is then applied to the test set. `knn()` breaks ties at random, so a seed is set before each fit to make the results reproducible.

```r
vars <- c("cylinders", "displacement", "horsepower", "weight")

x_train <- train[, vars]
x_test <- test[, vars]

train_mean <- sapply(x_train, mean)
train_sd <- sapply(x_train, sd)
```

```r
x_train_scaled <- scale(x_train, center = train_mean, scale = train_sd)
x_test_scaled <- scale(x_test, center = train_mean, scale = train_sd)

k_values <- c(1, 3, 5, 7, 9, 15, 25)

knn_errors <- sapply(k_values, function(k) {
  set.seed(639)
  pred <- knn(
    train = x_train_scaled,
    test = x_test_scaled,
    cl = train$mpg01,
    k = k
  )
  mean(pred != test$mpg01)
})

knn_results <- data.frame(
  K = k_values,
  Test_Error = knn_errors
)

knn_results
```

```
##    K Test_Error
## 1  1 0.05084746
## 2  3 0.08474576
## 3  5 0.08474576
## 4  7 0.09322034
## 5  9 0.09322034
## 6 15 0.09322034
## 7 25 0.09322034
```

```r
best_knn <- knn_results[which.min(knn_results$Test_Error), ]
best_knn
```

```
##   K Test_Error
## 1 1 0.05084746
```

For this split, the best tested value is $K = 1$, with test error 0.0508 (about 5.1%, or 6 misclassified test cars). For larger values of $K$, the test error is between 8.5% and 9.3%.

### 4.8.1 Model comparison

```r
model_results <- data.frame(
  Method = c("LDA", "QDA", "Logistic Regression", "Naive Bayes", paste0("KNN (K=", best_knn$K, ")")),
  Test_Error = c(lda_error, qda_error, logit_error, nb_error, best_knn$Test_Error)
)

model_results[order(model_results$Test_Error), ]
```

```
##                Method Test_Error
## 5          KNN (K=1) 0.05084746
## 2                 QDA 0.09322034
## 1                 LDA 0.10169492
## 3 Logistic Regression 0.10169492
```

```
## 4         Naive Bayes 0.10169492
```

All methods use the same training/test split, so their test errors can be compared directly. On this split, **KNN (K=1)** has the lowest test error, about 5.1%. Several points help put this ranking in context:

- **The differences are small in absolute terms.** The test set has only 118 cars. The best method misclassifies 6 of them, compared with 12 for the worst. A different random split could change the ranking, so a single split is weak evidence that one method is better. Cross-validation (Chapter 5) would give a more reliable comparison.
- **KNN with $K = 1$ is the most flexible model considered.** Its decision boundary follows individual training points, which gives it low bias but high variance. Its strong result here may partly reflect this particular split. For larger values of $K$, KNN's error is similar to that of the parametric methods.
- **LDA, logistic regression, and naive Bayes give identical confusion matrices.** All three produce a nearly linear boundary in these four predictors, and the classes are well separated by vehicle size. As a result, they classify every test car the same way. QDA allows a different covariance matrix for each class, which gives it a more flexible, curved boundary; on this split it misclassifies 11 test cars, compared with 12 for LDA.

Overall, every method misclassifies only about 5.1–10.2% of test cars. Cylinders, displacement, horsepower, and weight are strong predictors of whether a car has above-median mileage.

# 5 Chapter 9 - Exercise 3

We have seven observations in two dimensions.

```
svm_data <- data.frame(
  X1 = c(3, 2, 4, 1, 2, 4, 4),
  X2 = c(4, 2, 4, 4, 1, 3, 1),
  Class = factor(c("Red", "Red", "Red", "Red", "Blue", "Blue", "Blue")),
  Obs = 1:7
)

svm_data
```

```
##   X1 X2 Class Obs
## 1  3  4   Red   1
## 2  2  2   Red   2
## 3  4  4   Red   3
## 4  1  4   Red   4
## 5  2  1  Blue   5
## 6  4  3  Blue   6
## 7  4  1  Blue   7
```

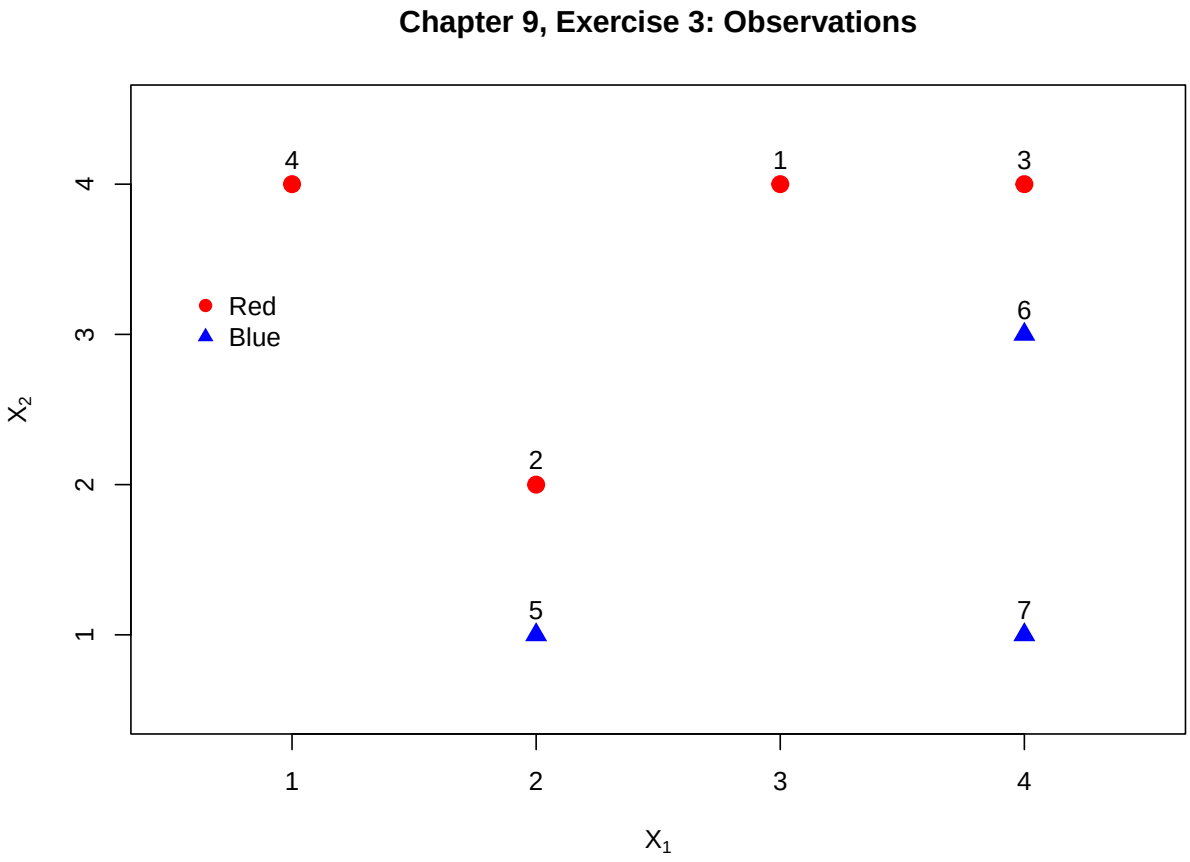
## 5.1 (a) Sketch the observations

A helper function draws the base plot. The axis limits leave room for the observation labels, and the points are colored by class.

```
plot_svm_base <- function(main) {
  plot(
    svm_data$X1,
    svm_data$X2,
    pch = ifelse(svm_data$Class == "Red", 19, 17),
    col = ifelse(svm_data$Class == "Red", "red", "blue"),
    cex = 1.4,
    xlim = c(0.5, 4.5),
```

```
    ylim = c(0.5, 4.5),
    xlab = expression(X[1]),
    ylab = expression(X[2]),
    main = main
  )
  text(svm_data$X1, svm_data$X2, labels = svm_data$Obs, pos = 3)
  legend(x = 0.55, y = 3.4, legend = c("Red", "Blue"),
         pch = c(19, 17), col = c("red", "blue"), bty = "n")
}
```

```
plot_svm_base("Chapter 9, Exercise 3: Observations")
```




## 5.2 (b) Optimal separating hyperplane

A convenient form of the maximal-margin hyperplane is

$$X_1 - X_2 = 0.5,$$

or equivalently

$$0.5 - X_1 + X_2 = 0.$$

**Why this is optimal.** Consider the quantity $X_1 - X_2$ for each observation:

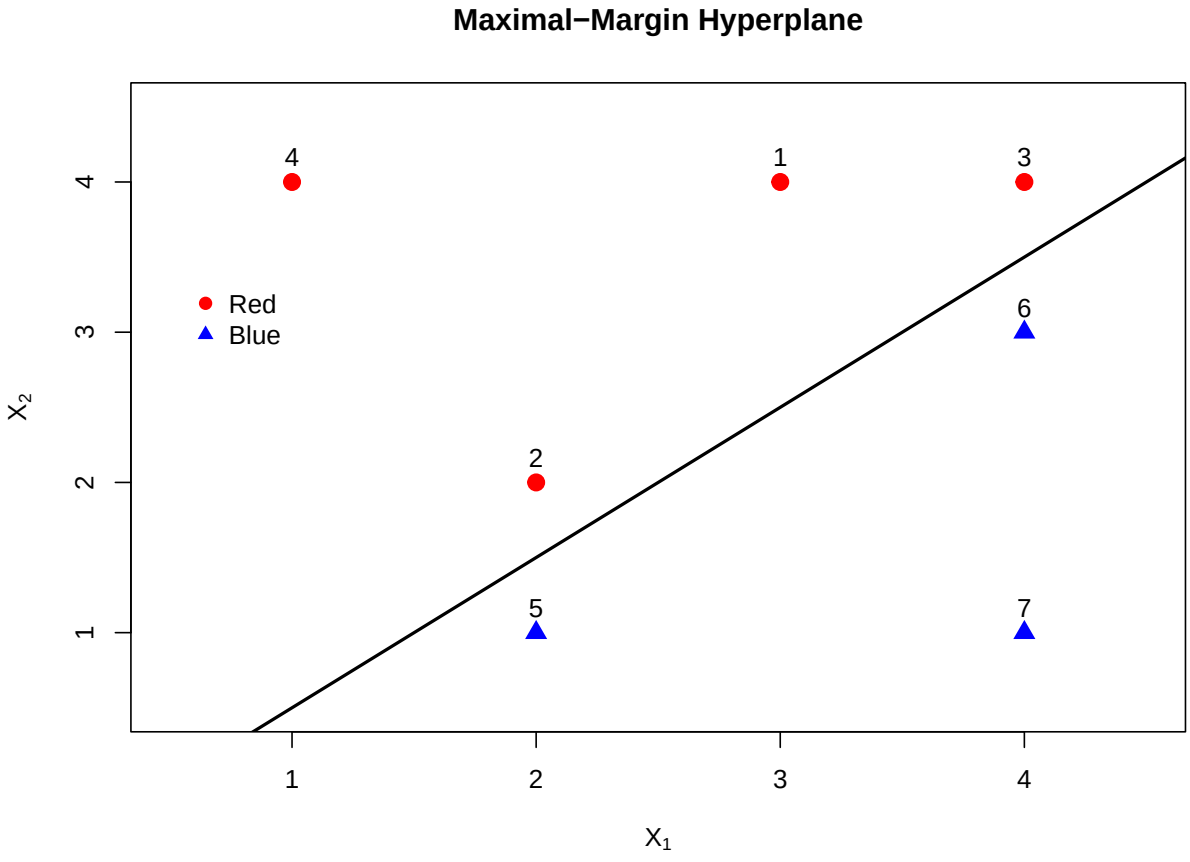
```
data.frame(Obs = svm_data$Obs, Class = svm_data$Class,
           X1_minus_X2 = svm_data$X1 - svm_data$X2)
```

```
##   Obs Class X1_minus_X2
## 1   1   Red          -1
## 2   2   Red           0
## 3   3   Red           0
## 4   4   Red          -3
## 5   5  Blue           1
## 6   6  Blue           1
## 7   7  Blue           3
```

Every red observation has $X_1 - X_2 \leq 0$, and every blue observation has $X_1 - X_2 \geq 1$. The closest points of the two classes are red observations 2 and 3, which lie on the line $X_1 - X_2 = 0$, and blue observations 5 and 6, which lie on the line $X_1 - X_2 = 1$. These two lines are parallel, and no line of a different slope can be placed farther from both pairs of points. The maximal-margin hyperplane therefore lies exactly halfway between them, at $X_1 - X_2 = 0.5$.

```
plot_svm_base("Maximal-Margin Hyperplane")

# X1 - X2 = 0.5  ->  X2 = X1 - 0.5
abline(a = -0.5, b = 1, lwd = 2)
```



## 5.3 (c) Classification rule

Using

$$\beta_0 = 0.5, \qquad \beta_1 = -1, \qquad \beta_2 = 1,$$

the classifier is

$$0.5 - X_1 + X_2 = 0.$$

Therefore:

- classify an observation as **Red** if $0.5 - X_1 + X_2 > 0$;
- classify it as **Blue** otherwise.

```
score <- 0.5 - svm_data$X1 + svm_data$X2
predicted <- ifelse(score > 0, "Red", "Blue")
data.frame(svm_data, score = score, predicted = predicted)
```

```
##   X1 X2 Class Obs score predicted
## 1  3  4   Red   1   1.5       Red
## 2  2  2   Red   2   0.5       Red
## 3  4  4   Red   3   0.5       Red
## 4  1  4   Red   4   3.5       Red
## 5  2  1  Blue   5  -0.5      Blue
## 6  4  3  Blue   6  -0.5      Blue
## 7  4  1  Blue   7  -2.5      Blue
```

## 5.4 (d) Margin

The two margin boundaries are

$$X_1 - X_2 = 0$$

and

$$X_1 - X_2 = 1.$$

The separating hyperplane $X_1 - X_2 = 0.5$ lies exactly halfway between them.

The margin $M$ is the perpendicular distance from the hyperplane to the nearest observations. The distance from a point $(x_1, x_2)$ to the line $0.5 - X_1 + X_2 = 0$ is

$$\frac{|0.5 - x_1 + x_2|}{\sqrt{(-1)^2 + 1^2}}.$$

For any of the support vectors, $|0.5 - x_1 + x_2| = 0.5$, so

$$M = \frac{0.5}{\sqrt{2}} = \frac{\sqrt{2}}{4} \approx 0.354.$$

The full width between the two dashed margin lines is $2M = 1/\sqrt{2} \approx 0.707$.

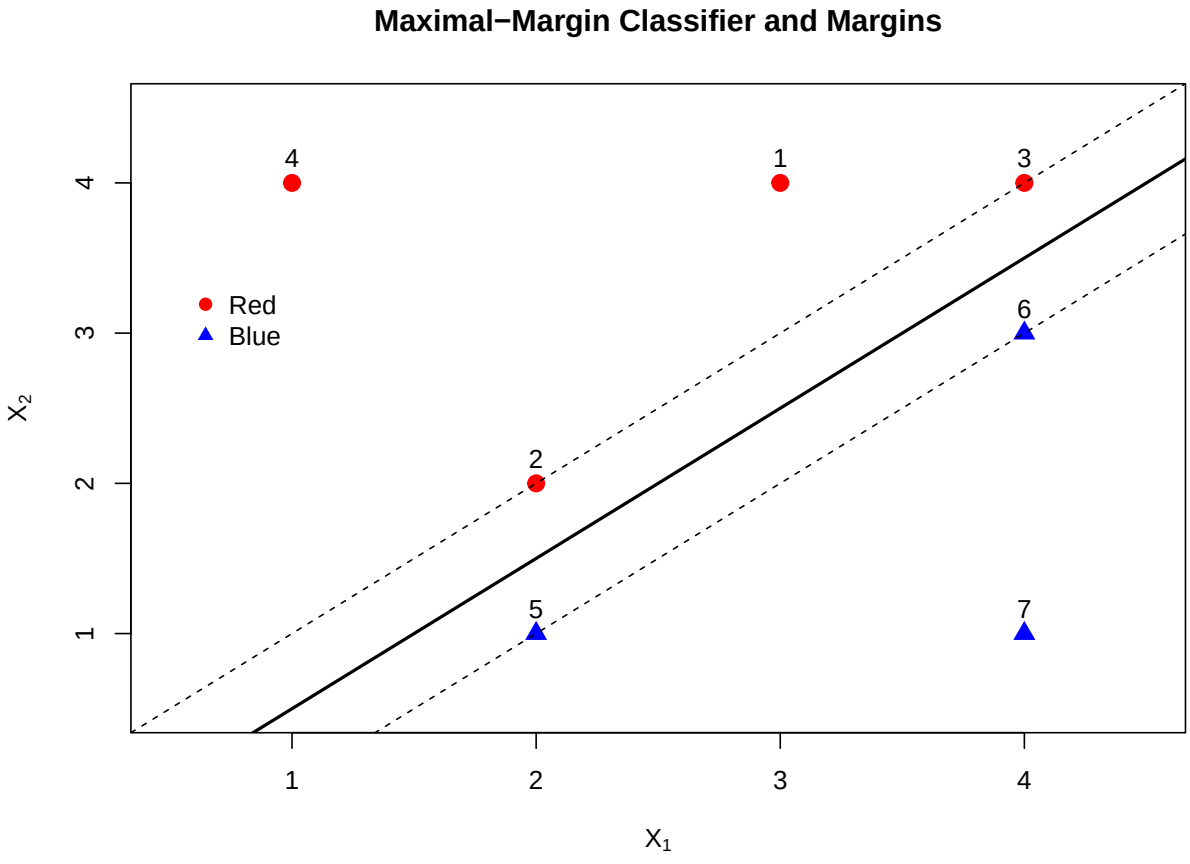
```r
dist_to_plane <- abs(0.5 - svm_data$X1 + svm_data$X2) / sqrt(2)
data.frame(Obs = svm_data$Obs, Class = svm_data$Class,
           distance = round(dist_to_plane, 3))
```

```
##   Obs Class distance
## 1   1   Red    1.061
## 2   2   Red    0.354
## 3   3   Red    0.354
## 4   4   Red    2.475
## 5   5  Blue    0.354
## 6   6  Blue    0.354
## 7   7  Blue    1.768
```

Observations 2, 3, 5, and 6 attain the minimum distance of 0.354.

```r
plot_svm_base("Maximal-Margin Classifier and Margins")

abline(a = -0.5, b = 1, lwd = 2)  # decision boundary
abline(a = 0, b = 1, lty = 2)     # red margin: X2 = X1
abline(a = -1, b = 1, lty = 2)    # blue margin: X2 = X1 - 1
```




## 5.5 (e) Support vectors

The support vectors are the observations lying on the two margin boundaries:

- Red: observations **2** and **3**, where $X_1 - X_2 = 0$
- Blue: observations **5** and **6**, where $X_1 - X_2 = 1$

```
svm_data[c(2, 3, 5, 6), ]
```

```
##   X1 X2 Class Obs
## 2  2  2   Red   2
## 3  4  4   Red   3
## 5  2  1  Blue   5
## 6  4  3  Blue   6
```

## 5.6 (f) Moving observation 7 slightly

Observation 7 is $(4, 1)$, so

$$X_1 - X_2 = 3.$$

It lies well inside the Blue side and is not a support vector. The maximal-margin hyperplane is determined by the observations closest to the boundary, namely observations 2, 3, 5, and 6. Therefore, a **small movement** of observation 7 that leaves it away from the margin will not change the maximal-margin hyperplane.